



UNINASSAU

Ciclo de Vida de Um Objeto, Abstração e Encapsulamento

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

Ciclo de Vida de Um Objeto

CICLO DE VIDA DE UM OBJETO

- Em POO, um objeto em Java passa por algumas etapas principais:

1. **Declaração** – Criamos a referência ao objeto.

- *Carro meuCarro;*

- Aqui, ainda não temos o objeto, só a referência.

2. **Instanciação** – Criamos o objeto em memória com **new**.

- *meuCarro = new Carro();*

3. **Utilização** – Chamamos métodos e acessamos atributos.

- *meuCarro.ligar();*

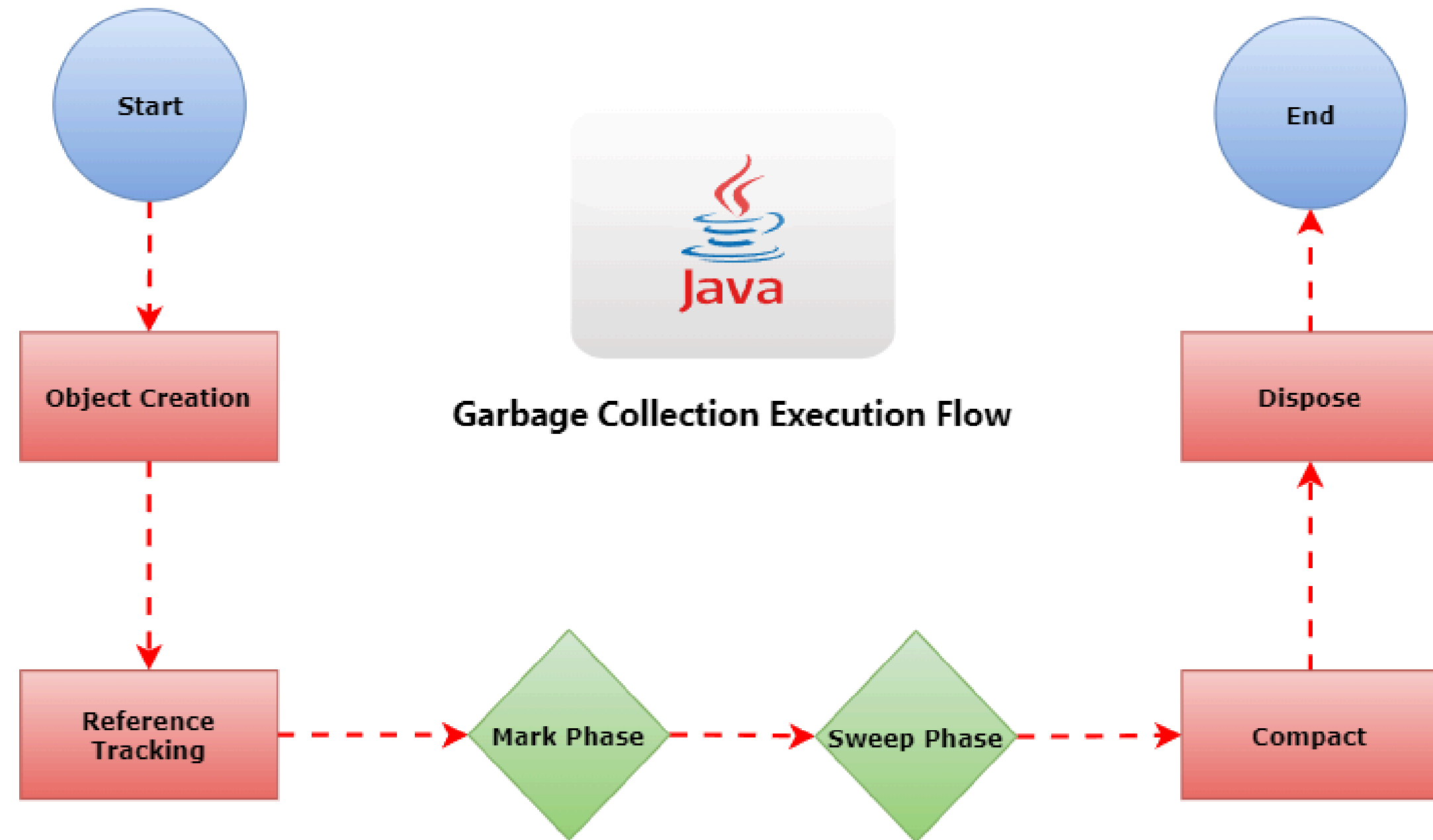
- *meuCarro.acelerar();*

4. **Finalização (Descarte)** – Quando o objeto não é mais usado, o **Garbage Collector** do Java libera a memória.

Não controlamos isso diretamente, mas podemos usar o método `finalize()` (deprecated, hoje em dia evitado) ou confiar no GC.

GARBAGE COLLECTOR

- o Garbage Collector é um processo de software que **automatiza o gerenciamento de memória**, liberando automaticamente o espaço de memória ocupado por objetos que não são mais referenciados ou alcançáveis pelo programa. Ele **substitui o gerenciamento manual de memória**, onde o programador precisaria alocar e desalocar memória explicitamente, prevenindo erros como vazamentos de memória (memory leaks) e garantindo que a memória seja utilizada de forma eficiente.



EXEMPLO:

```
class Carro {  
    void ligar() {  
        System.out.println("Carro ligado!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Carro c;           // Declaração  
        c = new Carro();    // Instanciação  
        c.ligar();          // Utilização  
        // Após o fim do programa, o objeto é liberado pelo GC  
    }  
}
```


UTILIZAÇÃO DE OBJETOS

- Após a criação, os objetos são utilizados através de suas referências para:
 - Acessar Atributos:

//Leitura de atributo

String nome = pessoa.getNome();

//Modificação de atributo

pessoa.setIdade(26);

UTILIZAÇÃO DE OBJETOS

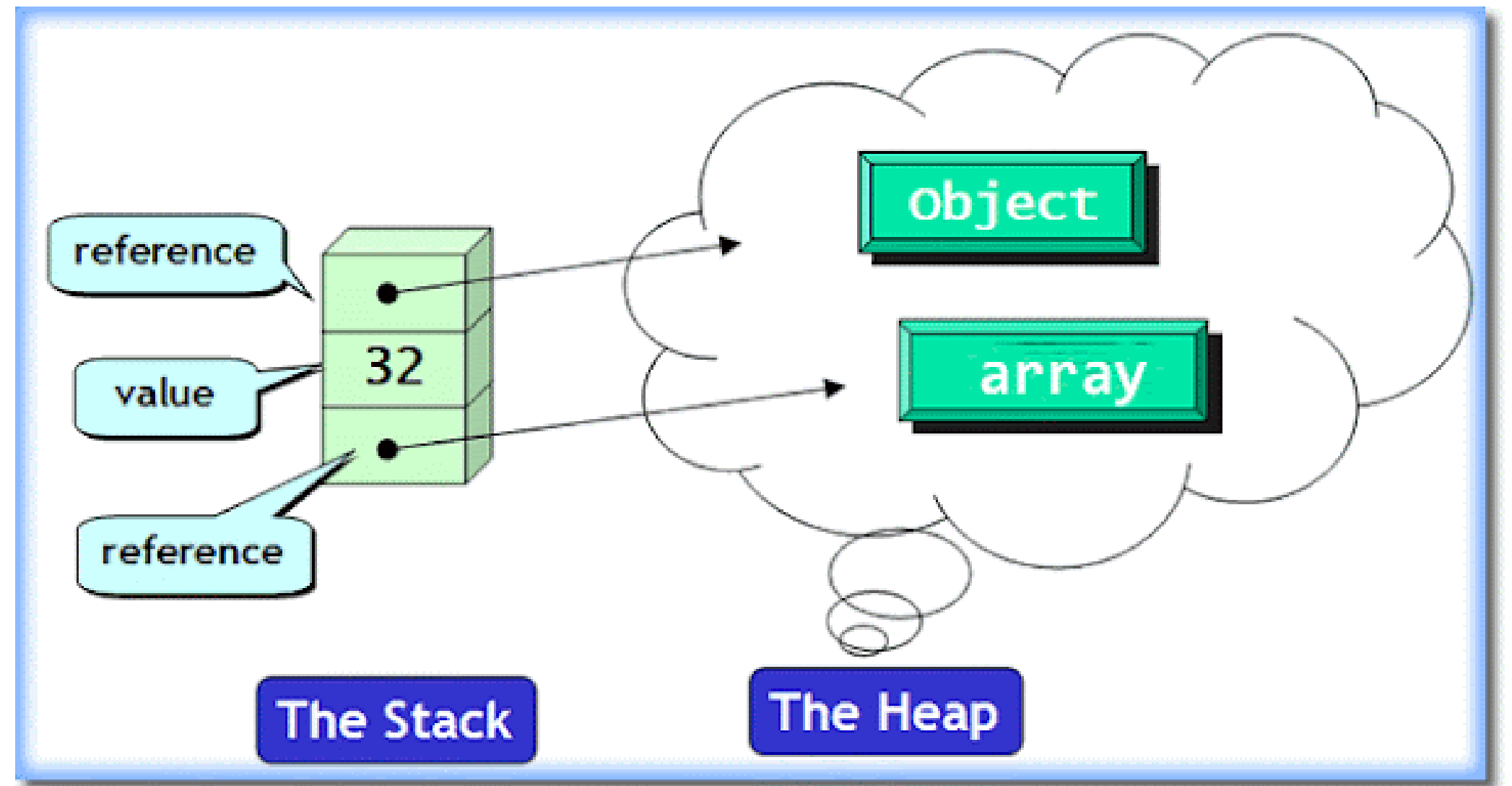
- Após a criação, os objetos são utilizados através de suas referências para:
 - Invocar Métodos:

```
// Chamada de método  
pessoa.fazerAniversario();  
pessoa.apresentar();
```

Durante esta fase, o objeto permanece na memória heap e pode ser acessado por qualquer parte do programa que possua uma referência válida a ele.

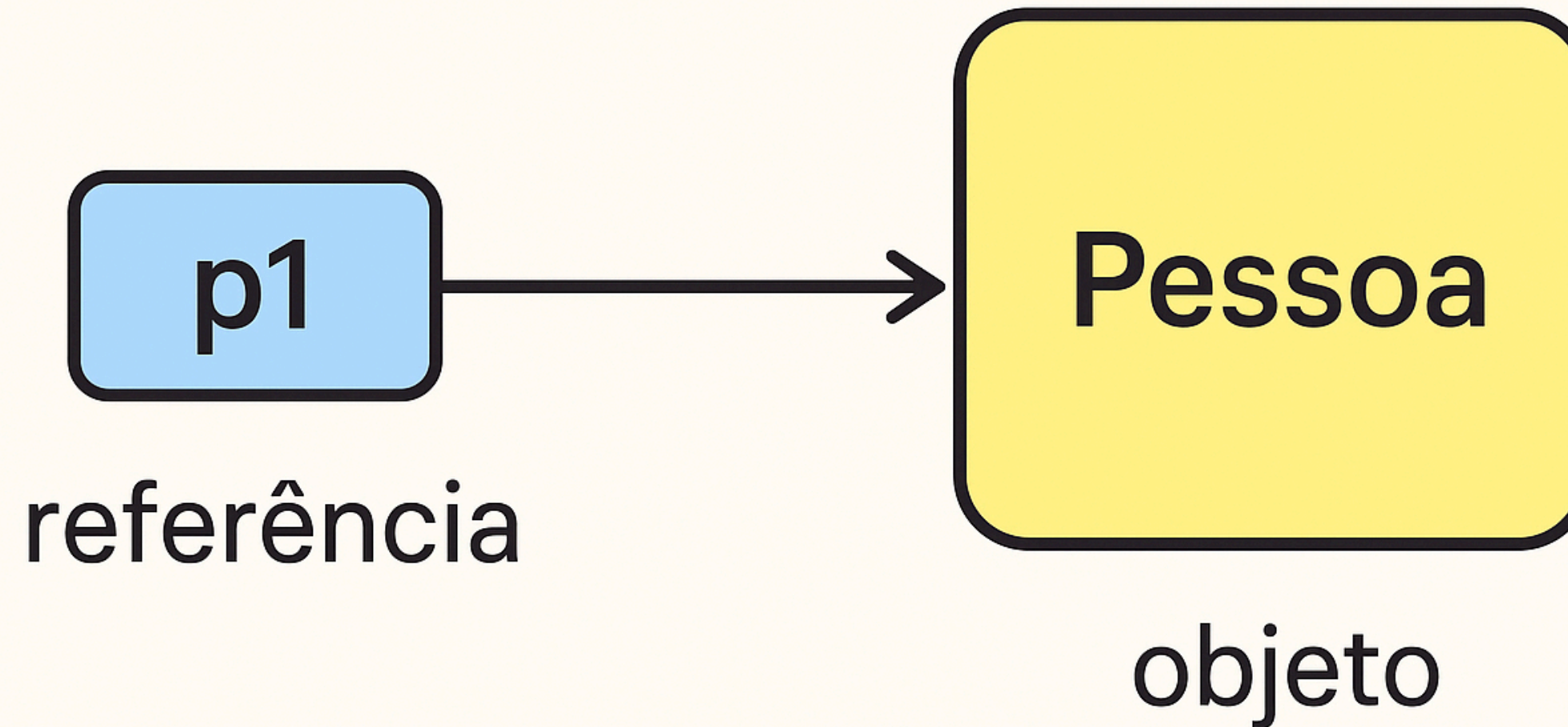
MEMÓRIA EM JAVA

- **Stack:** guarda referências
- **Heap:** guarda objetos reais
- **Diagrama visual:** referência → objeto na heap



EXEMPLO VISUAL

- Código: **Pessoa p1 = new Pessoa();**
- Exemplo: Caixa **p1** apontando para objeto **Pessoa**



DESTRUIÇÃO DE OBJETOS

- Em Java, a destruição de objetos é gerenciada pelo **Garbage Collector (GC)**, que **libera a memória ocupada por objetos que não são mais acessíveis**.

1. Um objeto se torna elegível para coleta quando:

- Todas as referências ao objeto são definidas como null
- O objeto sai do escopo (variáveis locais)
- Referências são reatribuídas a outros objetos

```
Pessoa pessoa = new Pessoa("João", 25);  
pessoa = null; // Objeto original agora é elegível para GC  
  
// Sugerindo execução do GC (não garante execução imediata)  
System.gc();  
  
System.out.println("Programa finalizado.");
```

EXEMPLO 1: DESTRUIÇÃO DE OBJETOS

```
class Pessoa {  
    private String nome;  
    private int idade;  
  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    @Override  
    protected void finalize() throws Throwable {  
        System.out.println("Objeto da pessoa " + nome + " foi coletado pelo Garbage Collector.");  
    }  
}
```

- Observações importantes:
 - O método finalize() está deprecated a partir do Java 9 e removido no Java 18.
 - Ele serve apenas para demonstrar didaticamente quando o GC coleta um objeto.

EXEMPLO 1: DESTRUIÇÃO DE OBJETOS

```
public class Main {  
    public static void main(String[] args) {  
        // Tornando um objeto elegível para GC  
        Pessoa pessoa = new Pessoa("João", 25);  
        pessoa = null; // Objeto original agora é elegível para GC  
  
        // Sugerindo execução do GC (não garante execução imediata)  
        System.gc();  
  
        System.out.println("Programa finalizado.");  
    }  
}
```

- O System.gc() não garante execução imediata do coletor de lixo, apenas sugere ao Java Virtual Machine (JVM).

EXEMPLO 2: DESTRUIÇÃO DE OBJETOS

- Hoje em dia, para demonstrar o descarte de objetos, pode-se usar o **Cleaner** da API do Java:

Class Pessoa:

```
import java.lang.ref.Cleaner;

class Pessoa {
    private String nome;
    private int idade;

    // Instância única do Cleaner (responsável por rodar ações de limpeza)
    private static final Cleaner cleaner = Cleaner.create();

    // Associação do objeto com a ação de limpeza
    private final Cleaner.Cleanable cleanable;

    // Classe interna que representa o "estado" a ser limpo quando o objeto for coletado
    static class Estado implements Runnable {
        private String nome;

        Estado(String nome) {
            this.nome = nome;
        }
    }
}
```


EXEMPLO 2: DESTRUIÇÃO DE OBJETOS

- Hoje em dia, para demonstrar o descarte de objetos, pode-se usar o **Cleaner** da API do Java:

Class Pessoa:

```
// Esse método será chamado automaticamente pelo Cleaner
@Override
public void run() {
    System.out.println("Objeto da pessoa " + nome + " foi coletado pelo Garbage Collector.
}

}

// Construtor da classe Pessoa
public Pessoa(String nome, int idade) {
    this.nome = nome;
    this.idade = idade;

    // Registrando o objeto no Cleaner, associando-o a uma ação de limpeza
    this.cleanable = cleaner.register(this, new Estado(nome));
}

}
```

@OVERRIDE

- O **@Override** é uma anotação em Java, e tem um papel muito importante quando trabalhamos com **herança, abstração e polimorfismo**.
- O que é o @Override?
- É uma **anotação (annotation) que indica que um método está sobrescrevendo (overriding)** um método de uma superclasse ou interface.
- Ajuda a garantir que você escreveu o método corretamente.
- Se você errar o nome ou a assinatura do método, o compilador vai gerar erro.

EXEMPLO 3: SEM @OVERRIDE (PODE CAUSAR ERRO SILENCIOSO)

```
class Animal {
    void emitirSom() {
        System.out.println("Som genérico de animal");
    }
}

class Cachorro extends Animal {
    // ERRO: método diferente (maiúscula diferente), mas o compilador não reclama
    void EmitirSom() {
        System.out.println("Au Au!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Cachorro();
        a.emitirSom(); // Vai imprimir: "Som genérico de animal"
    }
}
```

- Aqui o programador **acha** que sobrescreveu o método, mas na verdade criou um novo.

EXEMPLO 4: COM @OVERRIDE

```
class Animal {  
    void emitirSom() {  
        System.out.println("Som genérico de animal");  
    }  
}  
  
class Cachorro extends Animal {  
    @Override  
    void emitirSom() {  
        System.out.println("Au Au!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Cachorro();  
        a.emitirSom(); // Agora imprime: "Au Au!"  
    }  
}
```

- Com @Override, o compilador garante que o método realmente está sobrescrevendo o da classe Animal.

BENEFÍCIOS DO @OVERRIDE

- **Evita erros de digitação** (se você escrever errado o nome, dá erro de compilação).
- **Facilita leitura do código** (outros programadores sabem que é um método sobrescrito).
- **Garante polimorfismo correto** (o método da subclasse realmente substitui o da superclasse).

```
package ocp17.ch07;

public record BeardedDragon(boolean fun) {

     @Override
    public boolean fun() {
        return false;
    }
}
```

Abstração

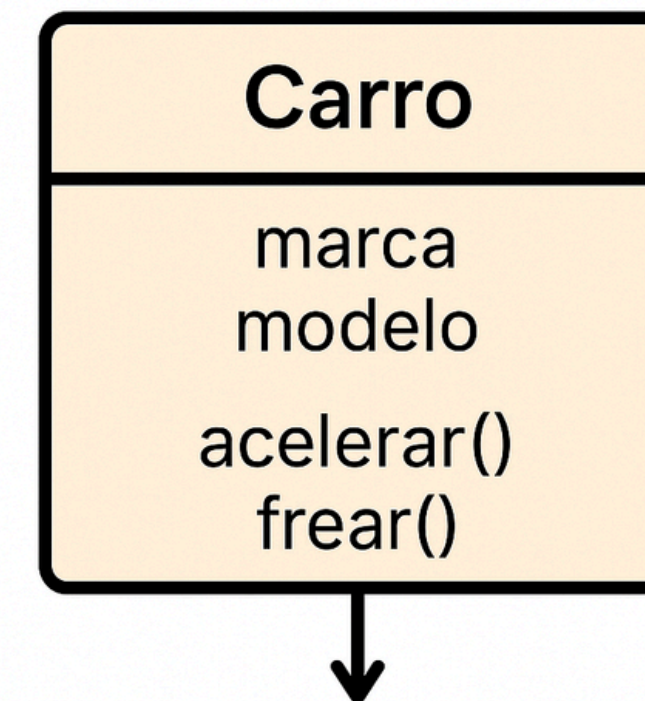
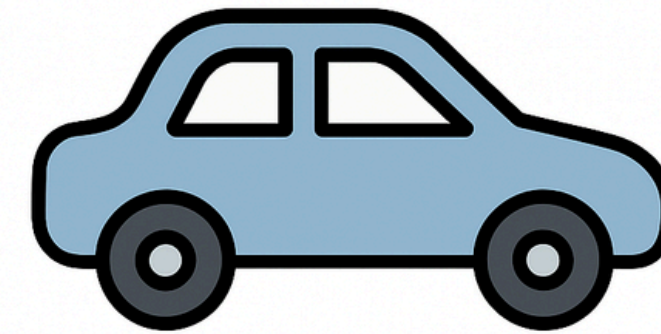
ABSTRAÇÃO

- A abstração é o processo de esconder detalhes de implementação e mostrar apenas o que é relevante.

Benefícios da Abstração:

- Reduz a complexidade, focando apenas no que é relevante
- Facilita a manutenção e evolução do código
- Promove o reuso através de hierarquias de classes
- Permite trabalhar em diferentes níveis de detalhe

Abstração

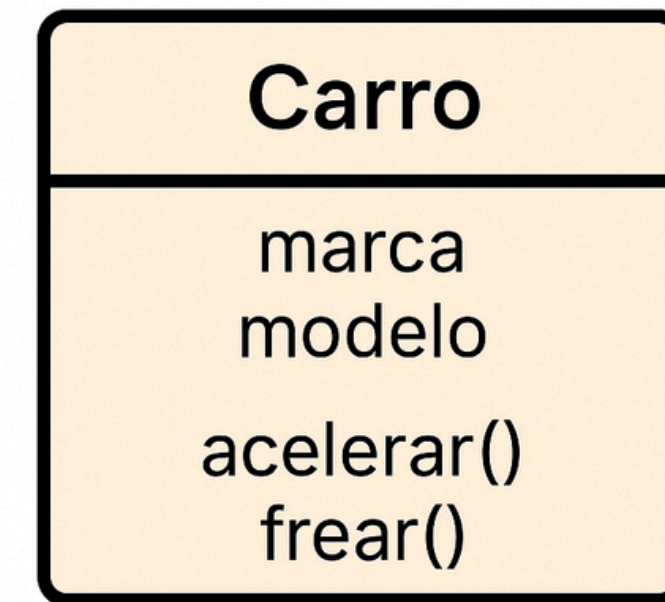
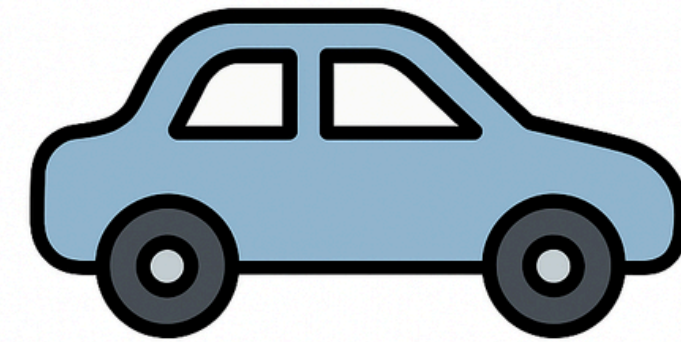


Extrair características
essenciais

ABSTRAÇÃO

- Exemplo: Uma *classe* **Carro** abstrai as características essenciais de um carro real, como **marca**, **modelo** e *métodos* como **acelerar()** e **frear()**, sem se preocupar com detalhes como o funcionamento interno do motor.

Abstração



Extrair características
essenciais

CLASSES ABSTRATAS E INTERFACES

Java oferece dois mecanismos principais para implementar abstração:

1) *// Classe abstrata*

```
abstract class Animal {  
    private String nome;  
  
    // Método concreto  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    // Método abstrato (deve ser implementado pelas subclasses)  
    public abstract void emitirSom();  
}
```

2) *// Interface*

```
interface Nadador {  
    void nadar(); // Método implicitamente público e abstrato  
}
```

CLASSES ABSTRATAS E INTERFACES

Característica	Classe Abstrata	Interface
Métodos	Concretos e abstratos	Abstratos (Java 8+ permite default)
Herança	Única	Múltipla
Uso ideal	Objetos relacionados	Comportamentos não relacionados

Exercício resolvido

EXERCÍCIO 1: CLASSE ABSTRATA E INTERFACE EM JAVA

Implemente um programa em Java que utilize interfaces para representar habilidades de animais.

1. Crie uma *interface* chamada *Nadador* com um *método* **nadar()**.
2. Crie uma *classe* **Cachorro** que:
 - Tenha um *atributo* **nome**.
 - Implemente **Nadador**.
 - Tenha um *método* **emitirSom()** que exiba "**Au Au!**".
3. Crie uma classe **Peixe** que:
 - Tenha um *atributo* **especie**.
 - Implemente **Nadador**.
4. Na **classe principal**, crie um *objeto* **Cachorro** e um *objeto* **Peixe**, chamando seus *métodos*.

EXERCÍCIO 1: CLASSE ABSTRATA E INTERFACE EM JAVA

1) Crie uma *interface* chamada *Nadador* com um *método* **nadar()** e uma *classe abstrata* **Animal**

```
// Interface
interface Nadador {
    void nadar();
}

// Classe Abstrata para servir de "molde" genérico de Animal
abstract class Animal {
    abstract void emitirSom();
}
```

EXERCÍCIO 1: CLASSE ABSTRATA E INTERFACE EM JAVA

2) Crie uma *classe* **Cachorro** que:

- Tenha um *atributo* **nome**.
- Implemente **Nadador**.
- Tenha um *método* **emitirSom()** que exiba "Au Au!".

```
// Classe Cachorro
class Cachorro extends Animal implements Nadador {
    private String nome;

    public Cachorro(String nome) {
        this.nome = nome;
    }

    @Override
    public void nadar() {
        System.out.println(nome + " está nadando feliz!");
    }

    @Override
    public void emitirSom() {
        System.out.println("Au Au!");
    }
}
```

EXERCÍCIO 1: CLASSE ABSTRATA E INTERFACE EM JAVA

3) Crie uma classe **Peixe** que:

- Tenha um *atributo* **especie**.
- Implemente **Nadador**.

```
// Classe Peixe
class Peixe implements Nadador {
    private String especie;

    public Peixe(String especie) {
        this.especie = especie;
    }

    @Override
    public void nadar() {
        System.out.println("O peixe da espécie " + especie + " está nadando no aquário!");
    }
}
```

EXERCÍCIO 1: CLASSE ABSTRATA E INTERFACE EM JAVA

4) Na **classe principal**, crie um *objeto Cachorro* e um *objeto Peixe*, chamando seus *métodos*.

```
// Classe Principal
public class Principal {
    public static void main(String[] args) {
        Cachorro cachorro = new Cachorro("Rex");
        Peixe peixe = new Peixe("Beta");

        cachorro.emitirSom();
        cachorro.nadar();

        peixe.nadar();
    }
}
```


EXERCÍCIO 1: CLASSE ABSTRATA E INTERFACE EM JAVA

Saída Esperada:

```
Au Au!
```

```
Rex está nadando feliz!
```

```
O peixe da espécie Beta está nadando no aquário!
```

EXERCÍCIO 1: CLASSE ABSTRATA E INTERFACE EM JAVA

O que foi feito:

- **Interface Nadador** → define o contrato **nadar()**.
- **Classe Abstrata Animal** → serve como base para animais que possuem som, mas não obriga herança para **Peixe**, já que ele não precisa de som.
- **Cachorro** → herda de **Animal** e implementa **Nadador**.
- **Peixe** → apenas implementa **Nadador**.
- **Principal** → cria *objetos* e chama os *métodos*.

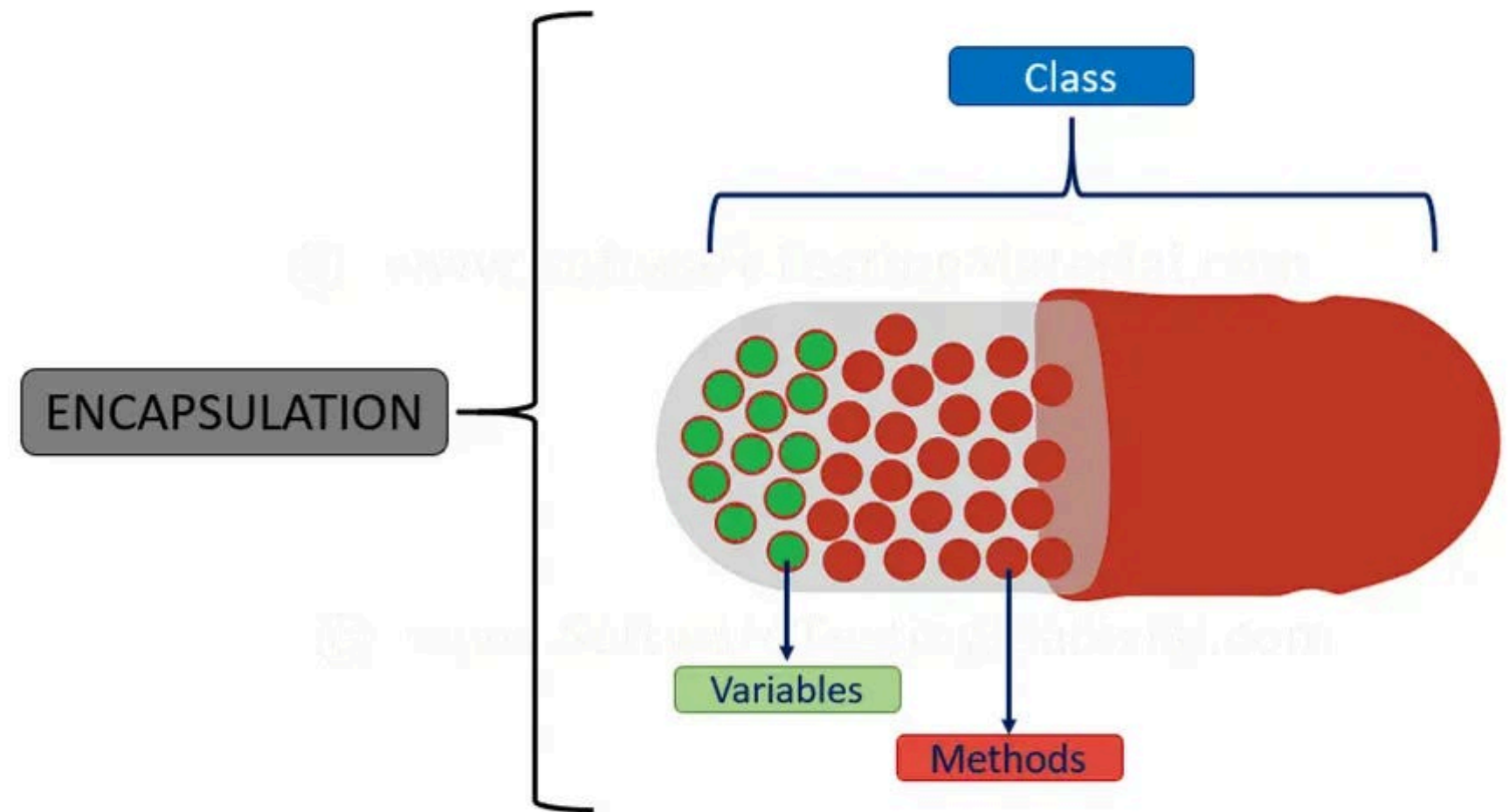
Encapsulamento

ENCAPSULAMENTO

- Encapsulamento é o princípio de ocultar os detalhes de implementação de uma classe e expor apenas o necessário.

Benefícios do Encapsulamento:

- Protege os dados contra acesso não autorizado
- Permite alterar a implementação sem afetar o código cliente
- Garante a validação de dados antes de modificá-los
- Reduz o acoplamento entre componentes do sistema



ENCAPSULAMENTO

- **Analogia:** Pense em um carro. Você interage com ele através de uma interface bem definida (volante, pedais, etc.), sem precisar conhecer os detalhes internos do motor ou sistema elétrico.



GETTERS, SETTERS E MODIFICADORES DE ACESSO

- O encapsulamento em Java é implementado principalmente através de:

Modificador	Visibilidade
private	Apenas na própria classe
default	Classes do mesmo pacote
public	Qualquer classe

GETTERS, SETTERS E MODIFICADORES DE ACESSO

- Getters e Setters:

```
private double saldo; // Atributo encapsulado

// Getter
public double getSaldo() {
    return this.saldo;
}

// Setter com validação
public void depositar(double valor) {
    if (valor > 0) {
        this.saldo += valor;
    } else {
        System.out.println("Valor inválido para depósito.");
    }
}
```

GETTERS, SETTERS E MODIFICADORES DE ACESSO

- Getters e Setters:

Se quisermos ter um **setter clássico**, ficaria assim:

```
public class ContaBancaria {  
    private double saldo; // Atributo encapsulado  
  
    // Getter  
    public double getSaldo() {  
        return this.saldo;  
    }  
  
    // Setter (simples, sem regras de negócio)  
    public void setSaldo(double saldo) {  
        this.saldo = saldo;  
    }  
}
```


GETTERS, SETTERS E MODIFICADORES DE ACESSO

- Getters e Setters:

Diferença:

- **setSaldo(double saldo)** → altera diretamente o valor (menos seguro, qualquer número pode ser colocado).
- **depositar(double valor)** → é um **setter especializado**, com validação para impedir valores inválidos.

```
public class ContaBancaria {  
    private double saldo; // Atributo encapsulado  
  
    // Getter  
    public double getSaldo() {  
        return this.saldo;  
    }  
  
    // Setter (simples, sem regras de negócio)  
    public void setSaldo(double saldo) {  
        this.saldo = saldo;  
    }  
}
```

Exercícios Propostos

EXERCÍCIO 1: CICLO DE VIDA

Crie uma *classe* **Livro** com *atributos* **titulo** e **autor**, e *métodos* **abrir()** e **fechar()**.

No main, mostre todas as fases do ciclo de vida de um objeto.

EXERCÍCIO 2: ABSTRAÇÃO

Crie uma *classe abstrata* **Funcionario** com *atributos* **nome** e **salario**.

Implemente dois filhos: **Gerente** e **Programador**, cada um com um *método* **calcularBonus()** diferente.

No *main*, crie uma lista de funcionários e mostre os bônus.

EXERCÍCIO 3: ENCAPSULAMENTO

Crie uma *classe* **Aluno** com *atributos privados* **nome** e **nota**.

Permita apenas que a nota seja alterada por um *método* **setNota(double nota)** que aceite valores apenas entre **0** e **10**.

No *main*, teste o comportamento.

Interstellar (2014)



As reservas naturais da Terra estão chegando ao fim e um grupo de astronautas recebe a missão de verificar possíveis planetas para receberem a população mundial, possibilitando a continuação da espécie. Cooper é chamado para liderar o grupo e aceita a missão sabendo que pode nunca mais ver os filhos. Ao lado de Brand, Jenkins e Doyle, ele seguirá em busca de um novo lar.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code